



Hewlett Packard
Enterprise

Automated Deployment of Docker Hosts on Bare-metal Infrastructure

Contents

Executive Summary	2
Business context: Composable Infrastructure	2
Containerization and Docker.....	3
Deployment of Docker hosts on bare-metal infrastructure.....	4
Walk-through.....	5
Step 1: Configure the HPE OneView server profile template	5
Step 2: Setup the OS build plan.....	7
Step 3: Create docker-machine from the command line	9
Step 4: Create the Docker client	11
Step 5: Execute Docker commands	11
Step 6: De-provision the server	13
Understand the provisioning.....	15
Best practices	15
Summary	16

Executive Summary

In today's Idea Economy, where a single idea can quickly become a new, industry-disrupting product or service, IT is expected to provide the infrastructure necessary to support the rapid deployment of new mobile and cloud-native products and services while continuing to maintain traditional IT applications. This forces IT to split resources across support for two different modes of infrastructure: one mode to support traditional applications where change is infrequent and reliability and stability are primary goals, and a second mode to support new apps and services where change is the norm and flexibility is the key requirement.

To address these competing needs, a new architectural vision of Composable Infrastructure is emerging. The concept is simple: physical and virtual resources can be provisioned to applications on demand, regardless of the style of application or the type or quantity of resource. Hewlett Packard Enterprise defines Composable Infrastructure as providing fluid pools of compute, storage, and fabric resources that can be aggregated and disaggregated on demand, abstracted through a single unified API, and controlled through the use of intelligent automation.

To support the DevOps need for infrastructure as code, Hewlett Packard Enterprise has partnered with noted industry visionaries, including Docker. Docker is an open platform for building, shipping, and running distributed applications. Docker containers spin up fast and provide a layer of isolation from other services running in containers. Docker also provides the ability to package an application with all its dependencies into a standardized unit for software development, guaranteeing that applications will always run the same, regardless of the environment. The integration of Docker with HPE OneView allows IT and DevOps to provision bare-metal container hosts directly from the Docker command line, in the same way they provision hosts in public clouds and virtualized environments. This approach provides deployment flexibility for enterprises that wish to use on-premises infrastructure to take advantage of the speed and consistency that containerization brings to application development.

This white paper shows IT architects, administrators, and DevOps developers how to deploy Docker hosts and execute containerized applications on bare-metal infrastructure.

Business context: Composable Infrastructure

In today's hyper-competitive business environment, it is easier than ever to disrupt industries and markets with new innovations. Cloud and mobile computing combined with the rise in social media have given business unprecedented opportunities to reach new customers with innovative products and services. In this Idea Economy, new businesses can quickly disrupt their industries with new ideas, often leaving more-established competitors scrambling to keep up.

As a result, the role of IT is changing. Lines-of-business put tremendous pressure on IT leadership to enable new applications and services—things like mobile apps, social, cloud, and the Internet-of-Things (IoT)—that are location-aware and drive real-time, personalized customer experiences and revenue opportunities. At the same time, IT must still manage the traditional applications that run the business—ERP, email, databases, etc.

The challenge is that these roles require two distinctly different approaches. In traditional IT, change is infrequent; infrastructure is static and applications spin a few times a year at most. With new-style applications, change delivery is continuous, with many companies spinning mobile and cloud-native applications daily, or even more frequently. To accomplish this, DevOps teams need dynamic infrastructure that can be deployed on demand.

Meeting these conflicting demands requires a hybrid or bi-modal approach to IT. While the public cloud can provide an option for some new applications, it can also present IT with enterprise architecture concerns over issues like security and data compliance. CIOs that want to host applications internally face the need for two separate infrastructures, one designed for reliability and stability to support mature applications and a second designed for agility and speed to support mobile and cloud-native applications. This is costly, complex, and is inconsistent with another directive that most CIOs confront: to simplify operations and reduce the overall cost of IT.

To address these competing needs, a new architectural vision is evolving that delivers a Composable Infrastructure. The concept is simple: physical and virtual resources can be provisioned to applications on-demand, regardless of the style of application or the type or quantity of resource.

For an infrastructure to be composable, it must be built around three basic principles:

- **Fluid resource pools**—fluid pools of disaggregated compute, storage and fabric resources that boot up ready for any physical or virtual workload and are composed instantly to support the needs of specific applications and services.
- **Software-defined intelligence**—template-driven workload automation to implement changes quickly and programmatically for frictionless operations
- **Unified API**—a single open API that provides a single line of code to abstract every element delivering true infrastructure as code

It's the combination of these three principles working together that defines composability. Anything less may provide some composable features, but will not provide the full benefits of hybrid/bi-modal IT from a single Composable Infrastructure.

Think of a Composable Infrastructure as a set of flexible building blocks that can be dynamically and automatically assembled and re-assembled to meet changing workload needs. A recent paper from Frost & Sullivan described it this way: "It's as if a child's set of Lego bricks came with the ability to replicate blocks as needed—and programmed-in instructions to configure themselves into a Ninja temple today and a working race car tomorrow."

Composable Infrastructure has the unique ability to run without being limited to one computing paradigm. It can support traditional and cloud-native apps in virtual machines, containers, or on bare metal. It can run anything and store anything. It also fully integrates with industry-standard tools like Chef, Docker, VMware®, OpenStack® and others, allowing IT operators and developers to continue leveraging the tools they use every day.

To learn more, read the white paper "[HPE Composable Infrastructure: Bridging traditional IT with the New Style of Business](#)".

Containerization and Docker

The Docker Machine Project is an open source project curated by Docker Inc. that creates an eco-system for running containerized applications.

Docker containers wrap up a piece of software in a complete file system that contains everything it needs to run: code, runtime, system tools, system libraries—anything that can be installed on a server. This guarantees that it will always run the same, regardless of the environment it runs in. It gives programmers, development teams, and operations engineers the common toolbox they need to take advantage of the distributed and networked nature of modern applications.

Like in a virtualized environment, containers allow multiple applications to gracefully share the same underlying infrastructure. Containers running on a single machine share the same operating system kernel, which allows them to start instantly and make very efficient use of system resources.

Docker uses resource isolation features of the Linux® kernel such as cgroups and kernel namespaces to allow independent "containers" to run within a single Linux instance. This allows containers to run on all major Linux distributions and cloud platforms. Security is achieved through isolation—Docker containers are isolated from each other and from the underlying infrastructure.

Containerized applications are packaged with their configurations and dependencies, making it easier to share apps from machine to machine. This simplifies collaboration within development teams and between developers and testers. Since developers don't need to worry about setting up and maintaining environments, they can put their full focus on creating new features, fixing issues, and shipping software.

A Docker host is an environment configured to run containers. Docker containers can run on any computer from a laptop to the cloud. To date, most large Docker environments have been deployed on the public cloud or inside virtual machines within enterprises. However, Docker containers running on top of bare-metal infrastructure, can reduce process latency.

Docker Machine lets users create Docker hosts on their computers, on cloud providers, and inside their data centers. It automatically creates hosts, installs Docker on them, then configures the Docker client to talk to them.

To meet application developer needs for instant access to infrastructure in a DevOps environment, IT operations are increasingly turning to Docker. For an example of DevOps deployment with Docker, see [Optimize DevOps with Docker and HPE BladeSystem](#).

Deployment of Docker hosts on bare-metal infrastructure

Hewlett Packard Enterprise is partnering with Docker to help enterprises run containers on bare-metal infrastructure with the same speed and flexibility as found in the public cloud. Using the unified Composable Infrastructure API, Hewlett Packard Enterprise has integrated Docker Machine with HPE OneView, allowing rapid provisioning of Docker hosts on bare-metal HPE infrastructure. This supports the DevOps need for infrastructure as code, letting developers and operations go from bare metal to container in minutes.

The Docker Machine Plugin for HPE OneView allows IT operations to automate physical provisioning of Docker infrastructure quickly and reliably. The plugin is a software integration that automates provisioning of a Docker environment onto Hewlett Packard Enterprise hardware infrastructure managed by HPE OneView. It uses HPE OneView templates and OS build plans to configure and provision bare-metal servers that host the Docker environment, in essence treating the infrastructure as code.

The driver for HPE OneView works in the same way as other Docker Machine drivers for cloud and virtual environments, such as Amazon Web Services or Oracle VirtualBox. For both IT and DevOps that equates to faster time to value, increased reliability and better DevOps alignment.

What follows is a brief overview of the solution, followed by a step-by-step walkthrough of the provisioning process.

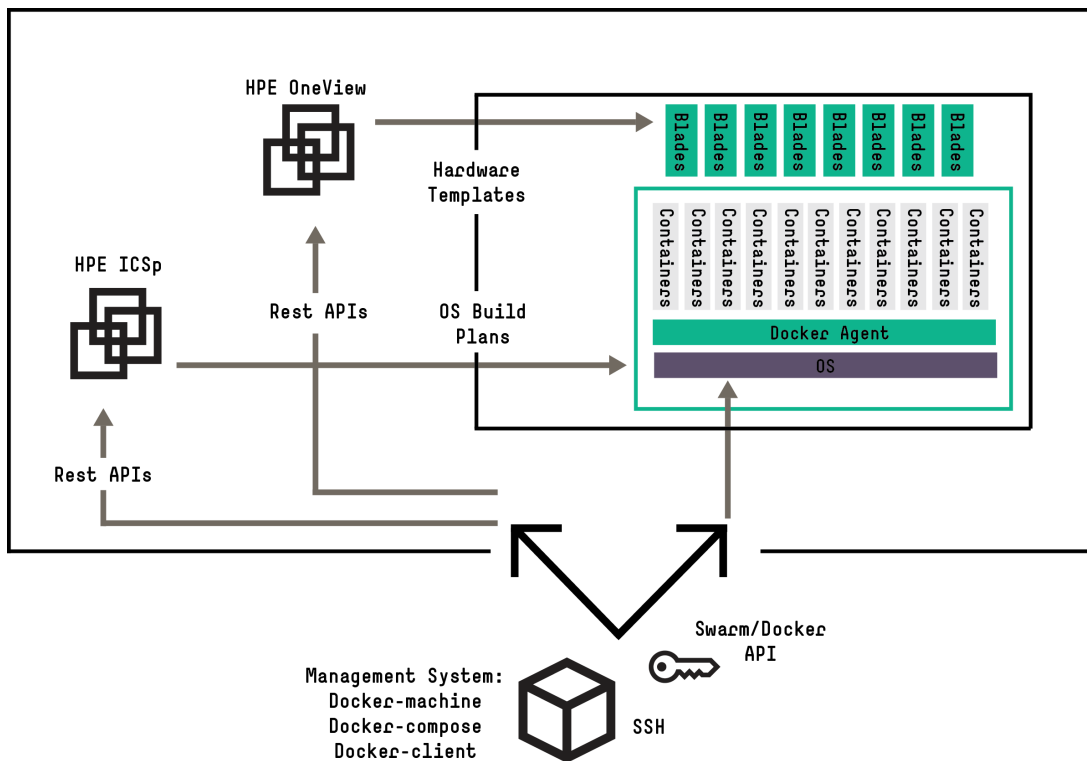


Figure 1. HPE OneView provisioning of Docker hosts with Docker tools

Docker Machine uses a plug-in mechanism to communicate with HPE OneView to manage hardware templates for blade provisioning and OS build plans for operating system deployment.

The Docker Machine plugin creates a host system by finding an available blade from HPE OneView, allocating that blade to the Docker host and then making that system available inside the HPE OneView OS deployment appliance (Insight Control server provisioning - ICSp). Once the OS build plan is complete, the Docker Machine plugin completes the software provisioning for the Docker engine. The Docker engine is installed on top of the operating system on the machine that is being managed by the Docker Machine mechanism, giving Docker-machine hooks to communicate with that provisioned system over SSH and over the Docker Swarm and Docker API ports.

At this point, the Docker client can spawn and start container applications on the host machine. Using Docker Swarm, you can group a cluster of blades or enclosures together to look like a single Docker agent. The end result is that applications development teams can now design workloads that can run on bare-metal infrastructure in your data center in a Docker-native way. Hewlett Packard Enterprise is the first to have the ability to do this in on-premises data centers.

Benefits for your business include:

- **Faster time to value**—Developers see faster build times, meaning less downtime and more productivity. Operations can provision entire application stacks from bare metal in minutes with no manual hardware configuration, reducing the time spent on deploying Docker environments.
- **Increased reliability**—Integration with HPE OneView ensures bare-metal servers are configured the same way every time, reducing complexity and management overhead for operations and allowing developers to focus on the application, not infrastructure.
- **Aligned with DevOps practices**—Automates the provisioning of physical infrastructure on-demand, from private, bare-metal resources using templates and enabling developers to treat infrastructure as code.

This approach will allow enterprises to take advantage of the speed and new features that containerization brings application development, all on their own infrastructure.

Walk-through

This walk-through demonstrates the new Docker Machine plugin for HPE OneView which is available on GitHub at: github.com/HewlettPackard/docker-machine-oneview

This walk-through will show the steps necessary to provision and de-provision a Docker machine on bare-metal infrastructure. The first two steps, creating templates for server profiles and operating system builds, are one-time activities that create re-useable templates.

Once templates are created, a developer can use a simple command-line interface to create a Docker host using Docker-Machine. The HPE OneView driver will use the specified template to provision the server infrastructure and ICsp will use the specified template to perform the operating system installation. Once the machine is provisioned, the developer can execute Docker commands. When the server is no longer needed, it can be de-provisioned with a simple command from the command line.

This walk-through assumes you have the following installed and running:

- An HPE OneView appliance with access to the HPE OneView UI through a browser. It must also have ICsp configured to deploy the OS of choice. You should have an account with Administrator privileges on the HPE OneView appliance and an account with Infrastructure Administrator role on the ICsp.
- A docker-machine, and docker-machine-oneview command line downloaded from the releases tab on the open source projects from github.com.

<https://github.com/docker/machine/releases>

<https://github.com/HewlettPackard/docker-machine-oneview/releases>

Place the executables on the system PATH environment variable.

- Available server enclosure and blade servers
- Licensed version of RedHat Enterprise Linux 7.1 and ISO images

Step 1: Configure the HPE OneView server profile template

The server profile template describes how the server will run in the data center, including firmware versions and settings, local and SAN storage configurations, boot settings, BIOS settings, network connections, etc.

Log in to your HPE OneView appliance using your username and password and start up the HPE OneView Dashboard in your browser. Navigate to “Server Profiles Templates” from the HPE OneView menu. A Profile Template can then be created for the Docker machine we will be building. Click the “Create server profile template” button on the upper left. The “Create Server Profile Template” popup window will open:

Edit docker1.8_server_templateGeneral?

General

Name

docker1.8_server_template

Description

docker-machine-oneview template

Server Profile

Server profile description

docker-machine-oneview template

Server hardware type

BL460c Gen9 1

Enclosure group

Houston

Affinity

Device bay

Firmware

Firmware baseline

managed manually

Connections

ID	Name	Address	Network	Port	Boot	
1	Priv_302		Priv_302_Set (network set)	FlexibleLOM 1:1-a	Primary	
	Type		Ethernet			
	Requested bandwidth		2.5 Gb/s			

Warning: This server profile template is in use by one or more server profiles.

OKCancel

Figure 2. HPE OneView server profile template configuration dialogue

Name your server template. In this case we’re using ‘docker1.8_server_template’, but you can use any name and description that work for your installation. Fill in the appropriate information for your own infrastructure, including Server Hardware, Enclosure Group, Firmware and BIOS settings, local and SAN storage settings, network settings, etc.

Optionally, when configuring the network connections, you can abstract the network name that will be used to communicate with the Docker Engine on the remote host with a name. Choose a name that can later be passed to the Docker Machine command line with the option ‘--oneview-public-connection-name’. By default, the second connection will be selected as the connected network for communicating to the Docker Machine.

Add Connection

Name

PublicNet1

Function type

Ethernet

Network

Prod_172_A

Port

Auto

Figure 3. HPE OneView Profile Template Connections dialogue

After filling in the template, click the “Create” button and your new template will be saved. This is a one-time activity, performed once for each server profile template to be created.

Step 2: Setup the OS build plan

The OS build plan defines the operating system that will run on a server. Once the Docker-machine has been created, provisioning an operating system onto the allocated hardware requires a working ICsp OS build plan.

ICsp comes out-of-the-box with pre-canned OS build plans that can be customized to your specific needs. In this example, we'll use one of the standard RedHat Linux 7.1 boot images located under OS Build Plans. You'll be modifying this image for your installation.

1. Select the RedHat 7.1 boot image (ProLiant OS – RHEL 7.1 x64 Scripted Install).

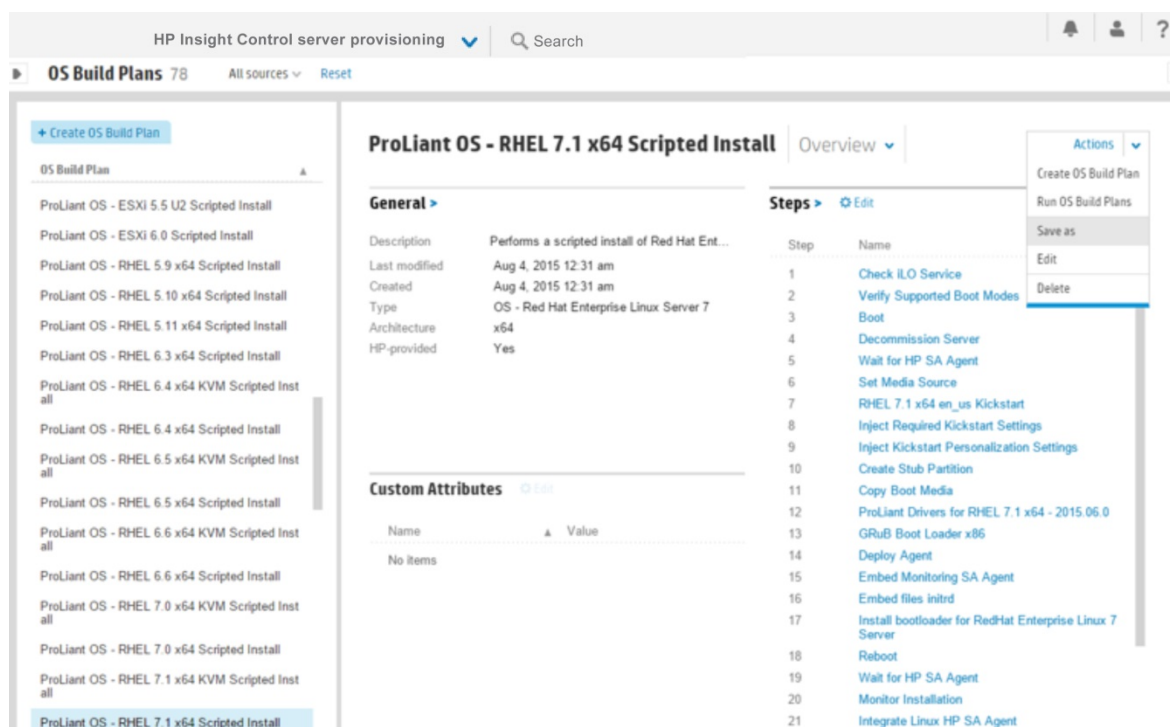


Figure 4. Select RedHat 7.1 build plan in HPE ICsp

This generic OS build plan contains all the steps for configuring RedHat and auto-booting it and auto-configuring it on the hardware.

2. Choose the action to save as a new OS build plan so it can be customized. The boot image can be named anything. In this case, we'll use RHEL71_DOCKER_1.8. You can customize this plan template with scripts that are specific to your environment.

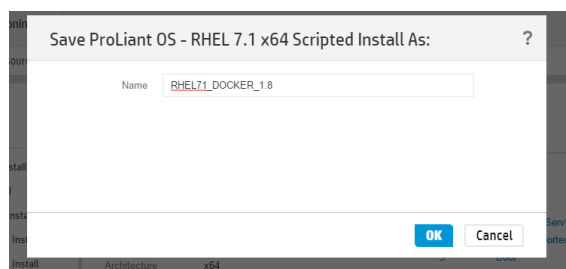


Figure 5. Save a new OS build plan

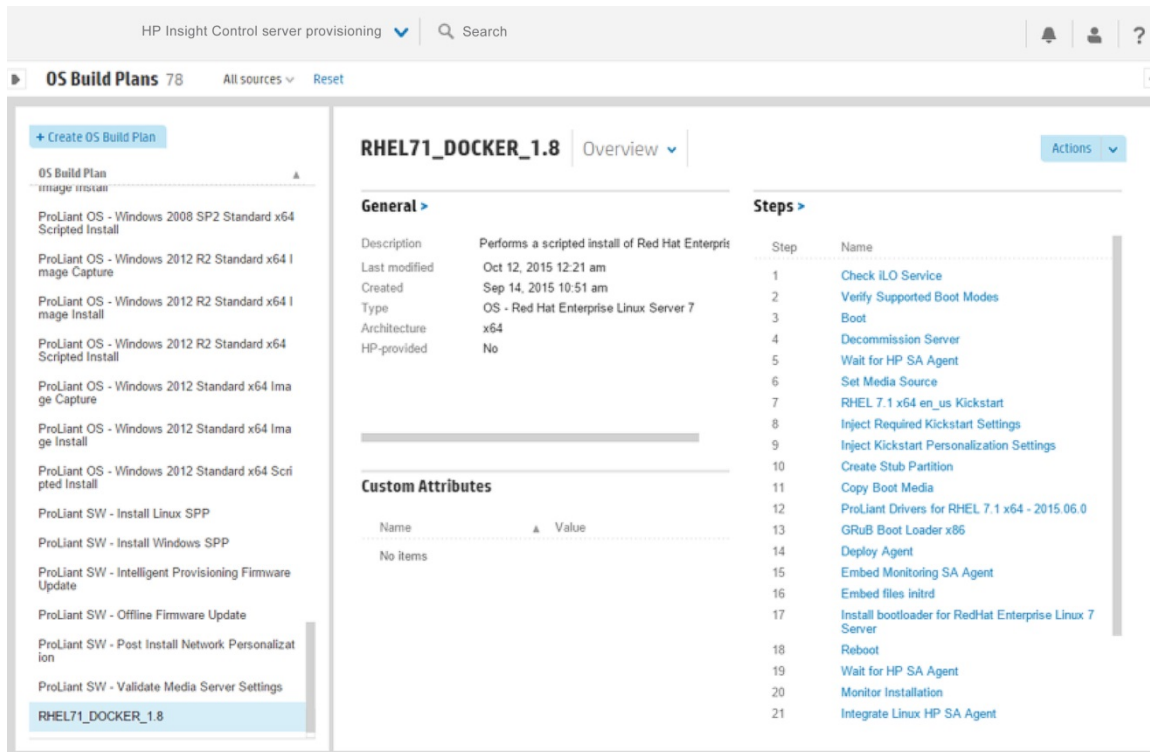


Figure 6. New customized OS Build Plan

3. On the RHEL71_DOCKER_1.8 build plan (step 25), add a bash script step to prepare the environment with basic user configuration and networking startup. The script should avoid fully provisioning Docker as this is managed by upstream Docker contributions to the Docker Machine project. Recommended script is provided on github: github.com/HewlettPackard/docker-machine-oneview/blob/master/scripts/docker_os_build_plan.sh

You can use this script to copy and paste the contents into the script step.

This script will pass several parameters to the OS build plan such as customizations to the Docker Machine command line to pass parameters including:

- @docker_user@—user name for the Docker user, used as the admin account with pseudo-privileges to install and run Docker commands
- @public_key@—the public key name that was automatically generated during the docker-machine create command
- @docker_hostname@—the hostname specified for the server
- @proxy_config@—the proxy configuration
- @proxy_enable@—when set to 'true', @proxy_config@ will be saved
- @interface@—the name of the public interface that should be used for Docker and machine traffic, this is dynamically determined based on connection-name and slotid options.

It is not recommended to hard-code these values; simply let the Docker Machine plugin update the values as needed.

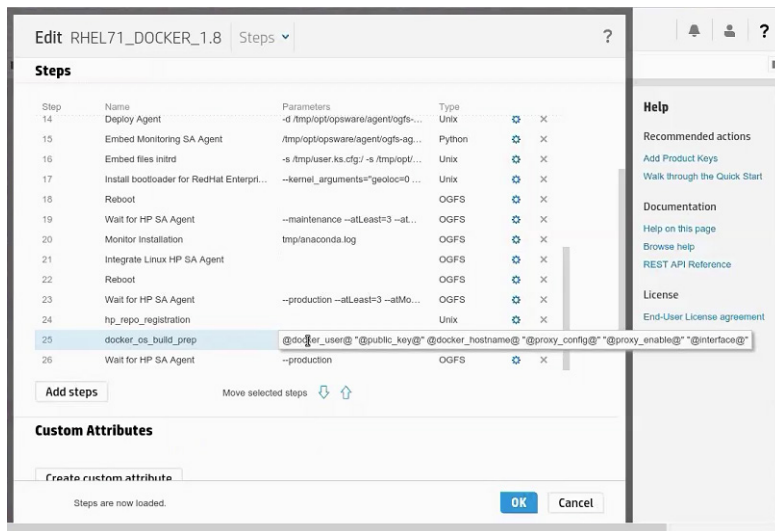


Figure 7. Customize the build step for custom OS Build Plan

The script will set up the user account to be used for Docker, set the public interface, set proxy configurations for the host operating system, and set up the account so that it can install Docker when the command is issued to do so.

You can choose to name this step 'docker_os_build_prep' or any other name that applies for your setup and environment. Refer to documentation provided on the docker-machine-oneview README.md on github.com for latest information on arguments and customizations that can be made in the step scripts contributed by community users.

4. Complete any additional setup scripts required to ensure specific local network requirements are met. This could include pointing to internal license servers or to internal repos such as your internal Linux or yum repositories. To meet these requirements, set up an additional build script to customize the OS configuration. This step is only needed where additional setup is required. An example is shown below.

Update /etc/yum.repos.d/ with special repos

```
cat > /etc/yum.repos.d/RedHat.repo << REPO
# base
[RedHat-7.1Server-x86_64-Server]
name=RedHat Linux 7.1Server - os - x86_64 - Server
baseurl=http://linuxcoe.corp.hp.com/LinuxCOE/RedHat-yum/7.1Server/en/os/x86_64
gpgcheck=0
# updates x86_64
[RedHat-7.1Server-x86_64-errata]
name=RedHat-7.1Server-x86_64-errata
baseurl=http://linuxcoe.corp.hp.com/LinuxCOE/RedHat-updates-yum/7.1Server/en/os/x86_64
gpgcheck=0
REPO
```

Creating an OS build plan is a one-time activity, performed once for each OS build plan template to be created. Once you have the OS build provisioning plan, you can use that plan in the Docker Machine command line to start and create the server.

Step 3: Create docker-machine from the command line

The templates are now in place and the provisioning process can begin. To create a Docker Machine, execute a docker-machine create command. Arguments include proxies, driver, OS build plan, and server-template.

In this case, we'll use the OS build plan values created in Steps 1 & 2 to produce the command line below.

```

master
docker-machine create \
> --engine-env HTTP_PROXY="$https_proxy" \
> --engine-env HTTPS_PROXY="$https_proxy" \
> --engine-env NO_PROXY="$no_proxy" \
> --driver oneview \
> --oneview-os-plan "RHEL71_DOCKER_1.8" \
> --oneview-server-template "docker1.8_server_template" \
> docker-ip-node-04-06

```

Figure 8. Single command line to provision your entire infrastructure

Enter the host name for the machine, in this case 'docker-ip-node04-06.' You can use any name that makes sense for your environment. Arguments for endpoint URLs, user name, and password for HPE OneView, ICsp, and ILO credentials can be provided as environment variables or additional command line arguments.

The create command triggers the following:

- Docker-machine generates SSH keys that will be used to configure the SSH user specified on the command line. If no user is specified, the docker user is used as the default
- HPE OneView creates a new server profile using the specified server profile template. This brings the server up to a state for intelligent provisioning
- The server is added to HPE ICsp
- HPE ICsp performs OS provisioning using the specified OS build plan
- The docker-engine is fully installed on the server by docker-machine

When the physical configuration is complete, review the results in the ICsp provisioning dashboard to see that everything is configured correctly.

The screenshot displays the HP Insight Control server provisioning dashboard. The top navigation bar includes 'HP Insight Control server provisioning', a search bar, and user icons. The main section is titled 'Servers 15' and shows a list of servers on the left. The selected server, 'docker-host0406-miqui', is shown in the main panel with its 'Overview' tab active. The 'General' section displays the following details:

- Host name:** docker-host0406-miqui
- Server state:** OK
- Description:**
- Hardware model:** PROLIANT BL460C GEN9
- Operating system:** Red Hat Enterprise Linux Server 7 X86_64
- Deployment interface:**
- ILO IP address:**
- Rack:** IC04
- Enclosure Bay:** 6
- Network:**

The 'History' section shows a table of provisioning tasks:

Provisioning Task	Job Start Time
RHEL71_DOCKER...	2015-10-28 09:48
ProLiant HW - Boot...	2015-10-28 09:38
Register server for i...	2015-10-28 09:37

The 'Custom Attributes (7)' section lists the following attributes:

- docker_hostname
- docker_user
- interface
- proxy_config
- proxy_enable
- nublic_in

The right sidebar contains links for 'Help', 'Documentation', 'Help on this page', 'Browse help', 'REST API Reference', 'License', and 'End-User License agreement'.

Figure 9. HPE ICsp dashboard after host provisioning with Docker Machine

At this point, the server is fully provisioned. A Docker host has been created on bare-metal infrastructure. You can see from the HPE OneView display that the server was booted, registered with the ILO, brought into Intelligent Provisioning, and the OS build plan was applied.

Returning to the command line and running a Docker list command (ls) shows that the server is available to host workloads.

```

master
docker-machine ls
NAME                ACTIVE  DRIVER  STATE  URL                SWARM
docker-ip-node-04-05 *        oneview Running tcp://16.85.3.140:2376
docker-ip-node-04-06 *        oneview Running tcp://16.85.3.13:2376

```

Figure 10. List your provisioned hosts

Step 4: Create the Docker client

Executing an env command generates a string that can be used to integrate the Docker-machine with the Docker command-line interface.

```

master
docker-machine env docker-ip-node04-05
export DOCKER_TLS_VERIFY="1"
export DOCKER_HOST="tcp://16.85.3.179:2376"
export DOCKER_CERT_PATH="/home/eraigosa/.docker/machine/machines/docker-ip-node04-05"
export DOCKER_MACHINE_NAME="docker-ip-node04-05"
# Run this command to configure your shell:
# eval "$(docker-machine env docker-ip-node04-05)"

```

Figure 11. Configure an active host

Executing the env command pulls configuration information from HPE OneView and creates a string that can be used to configure the command line.

The docker eval command allows you to communicate with the Docker client to the remote docker-engine service provisioned in the previous step by HPE OneView.

Executing an eval command with the docker-machine env as a sub-command will cause the environment variables generated above by the env command to be set for the current environment.

```

master
eval "$(docker-machine env docker-ip-node04-05)"

```

Figure 12. Activate a host

The environment is now ready to execute Docker commands.

Step 5: Execute Docker commands

You can now execute Docker commands. These are the same commands you've been executing in the cloud, but now they're running against bare metal.

Some examples:

1. Docker info allows you to see what kind of physical system was provisioned.

```

zsh: correct 'info' to '.info' [nyae]? n
Containers: 0
Images: 0
Storage Driver: devicemapper
Pool Name: docker-253:1-84086800-pool
Pool Blocksiz: 65.54 kB
Backing Filesystem: xfs
Data file: /dev/loop0
Metadata file: /dev/loop1
Data Space Used: 1.821 GB
Data Space Total: 107.4 GB
Data Space Available: 50.26 GB
Metadata Space Used: 1.483 MB
Metadata Space Total: 2.147 GB
Metadata Space Available: 2.146 GB
Udev Sync Supported: true
Deferred Removal Enabled: false
Data loop file: /var/lib/docker/devicemapper/devicemapper/data
Metadata loop file: /var/lib/docker/devicemapper/devicemapper/metadata
Library Version: 1.02.93-RHEL7 (2015-01-28)
Execution Driver: native-0.2
Logging Driver: json-file
Kernel Version: 3.10.0-229.el7.x86_64
Operating System: Red Hat Enterprise Linux Server 7.1 (Maipo)
CPUs: 56
Total Memory: 251.6 GiB
Name: docker-ip-node04-05
ID: E4Y7:XHT5:R4QI:EF46:PHB3:ZUIY:DTG0:P2PT:HUIM:ZJZP:DWIH:NTQQ
Labels:
provider=oneview

```

Figure 13. Get the current activated host information

From this command, you can see that the command line is attached to a server, that the server is running RedHat 7.1, it's a system with 56 CPUs and 251.6 GB of memory. You can also see that the command line is attached to a physical machine.

2. Pull a Docker container image

```

master
$ docker pull busybox
Using default tag: latest
latest: Pulling from library/busybox
28779c628bf0: Pull complete
0292957ealdd: Pull complete
Digest: sha256:3ebe07818fc2a8001cbb672b878ab0b81f047066093bb9c3f05600514710b921
Status: Downloaded newer image for busybox:latest

```

Figure 14. Pull a test image

This command pulled down the container image for busybox.

3. Run a container image. Start a busybox shell. From the shell, run a top command to display the running processes on the container. Run a ps command to list the containers.

The Docker command talks to the remote system to start a new container. This is made possible by the interaction of HPE OneView's APIs with docker-machine, which allows the docker command line interface to talk to the docker-engine service.

```

master
$ docker run -it busybox /bin/sh
/ # top
Mem: 4880524K used, 258953024K free, 17688K shrd, 2876K buff, 2794224K cached
CPU:  0.0% usr  0.0% sys  0.0% nic 99.9% idle  0.0% io  0.0% irq  0.0% irq
Load average: 0.00 0.01 0.05 2/655 5
  PID  PPID  USER      STAT  VSZ %VSZ CPU %CPU COMMAND
   1     0 root        S    1200  0.0  45  0.0 /bin/sh
   5     1 root        R    1188  0.0  47  0.0 top
/ # ps -ef
PID  USER      TIME  COMMAND
  1  root          0:00 /bin/sh
  6  root          0:00 ps -ef

```

Figure 15. Run the test image

4. Download and run a container from Docker Hub. For this example, the Jenkins container, which can be used to start a web site, can be found at: hub.docker.com/_/jenkins/



The screenshot shows the Jenkins Docker Hub page. It includes the Jenkins logo, the title "How to use this image", and two code blocks with Docker commands. The first command is `$ docker run -p 8080:8080 jenkins`, and the second is `$ docker run --name myjenkins -p 8080:8080 -v /var/jenkins_home jenkins`. The page also mentions that the workspace is stored in `/var/jenkins_home` and that the volume for the "myjenkins" container will be persistent.

Figure 16. Try the Jenkins Docker image

You can obtain many different types of apps from docker.io or create your own and use from your own private registry.

Use the run command to run the container for Jenkins.

```
~ master
docker run -p 80:8080 jenkins
```

Figure 17. Run a Jenkins container on an HPE OneView managed system

-p allows you to specify the port number. We'll use port 80 and direct that to the internal port which is 8080. The name of the container to run is Jenkins.

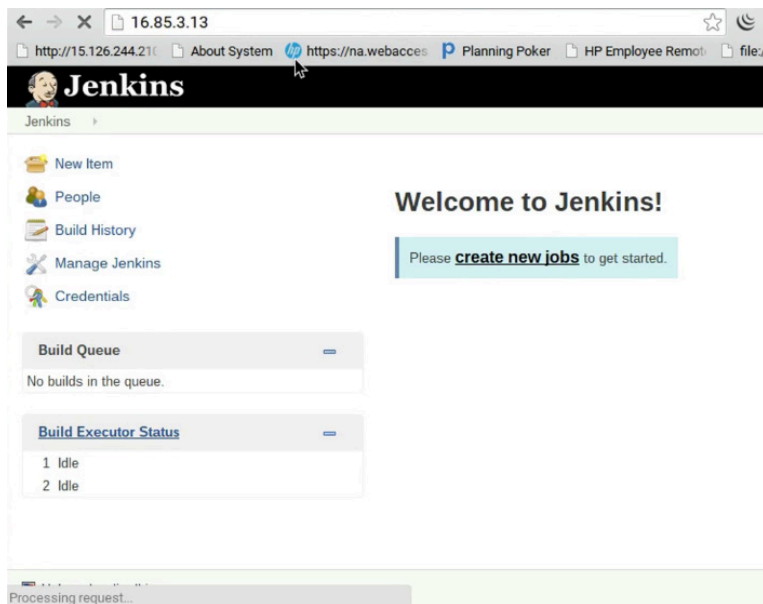


Figure 18. The Jenkins application running with an HPE OneView managed system

The Jenkins program is running!

These are just a few examples of the Docker commands that can be performed from physical machines. Any command that can be executed using docker-machine in the cloud can now be performed against physical resources.

Step 6: De-provision the server

When the Docker host is no longer needed, it can be de-provisioned with a single command. Execute the docker-machine remove (rm) command to remove the server from the Docker host and power down.

```
~/git/HewlettPackard/docker-machine-oneview > wenlock ov dev
docker-machine rm docker-ip-node04-06
```

Figure 19. Remove the host machine from HPE OneView

The docker-machine plugin communicates with HPE OneView and HPE ICsp to de-provision and then power down the server.

```
~/git/HewlettPackard/docker-machine-oneview > wenlock ov dev
docker-machine rm docker-ip-node04-06
Stop ... docker-ip-node04-06
Problem shutting down gracefully : exit status 255
Working on power state.
Powering Off server H0UEGSL-C7000-04, bay 6 for 2M2536042H.
Working on power state,3%, Power off server: H0UEGSL-C7000-04, bay 6.
Working on power state,100%, Successfully powered off server: H0UEGSL-C7000-04, bay 6..
Power Task Execution Completed
Waiting on, Delete, 10%, Delete, Set administrative server power lock for the server hardware H0UEGSL-C7000-04, bay 6
Waiting on, Delete, 50%, Delete, Remove server personality and connections.
Waiting on, Delete, 50%, Delete, Remove server personality and connections.
Waiting on, Delete, 50%, Delete, Remove server personality and connections.
Waiting on, Delete, 100%, Successfully deleted server profile: docker-ip-node04-06., Release administrative server power lock for the
server hardware H0UEGSL-C7000-04, bay 6
Task, Delete, completed
Successfully removed docker-ip-node04-06
```

Figure 20. HPE OneView has returned the server to the pool

The de-provisioning step removed the server from ICsp and then deleted it from HPE OneView. The HPE OneView Server Profile screen shows that the server no longer exists.

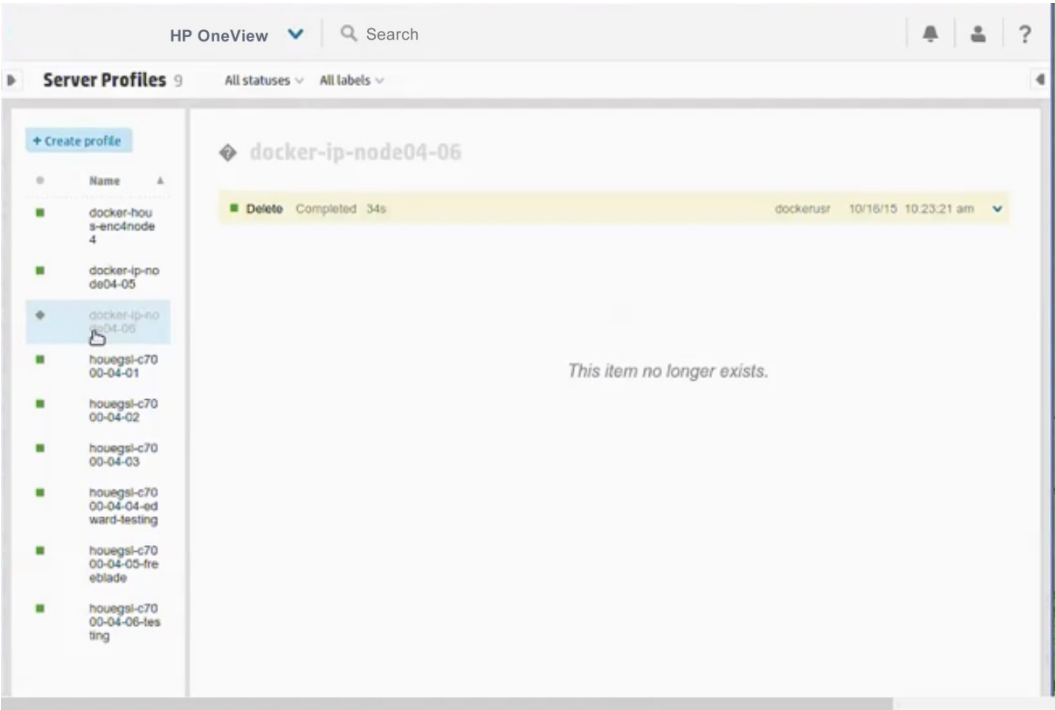


Figure 21. HPE OneView now shows server profile is no longer assigned

When the Docker remove command is complete, the OneView Server Hardware screen shows that the physical blade is powered off and has 'No Profile Applied.'

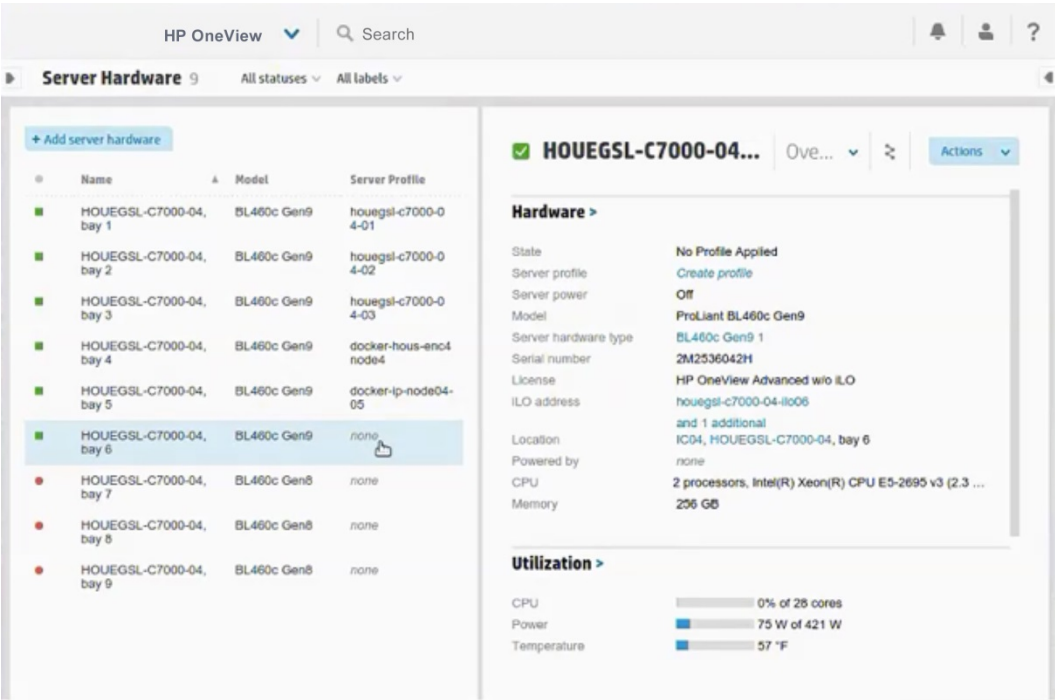


Figure 22. HPE OneView shows the server available again for provisioning

The physical blade is now ready and available again for provisioning.

Understand the provisioning

The HPE OneView driver performs the following steps during the provisioning process:

1. **Locates and allocates compatible server hardware** from the available pool of servers managed by HPE OneView, using the HPE OneView search REST API.
2. **Creates new instances from the selected Server Profile Template** using the HPE OneView REST API:
 - a. **Firmware, BIOS, and local storage configuration:** HPE OneView uses the embedded HPE Intelligent Provisioning (IP), a Linux pre-boot environment embedded on the iLO baseboard management card (BMC) processor, and the embedded SmartStart tooling.
 - b. **Network connectivity:** HPE OneView uses the Virtual Connect networking platform to dynamically provision network connectivity for servers from a pool of network connections.
 - c. **SAN storage:** HPE OneView automates SAN storage provisioning for HPE 3PAR SAN storage systems and Fibre Channel switches including Brocade, Cisco, and HPE Networking.
3. **Deploys the selected OS once the hardware provisioning is complete:**
 - a. The OS deployment network connection in the server profile is configured to PXE boot to the OS deployment server.
 - b. The server hardware is powered on and PXE boots.
 - c. The server registers with the OS deployment server.
 - d. The HPE OneView driver detects the server is registered and schedules the OS installation using the HPE OneView REST API for OS deployment (Insight Control server provisioning).
4. **Configures the host networking:**
 - a. IP address, subnet, hostname, etc. for different network adapters and virtual network adapters, allowing you to build sophisticated private networking arrangements.

Best practices

Below are some recommended best practices for the deployment of Docker on bare-metal infrastructure.

Run Docker with an OS service account

Running Docker with a service account rather than the root account can give you more flexibility for securing and managing OS level security for Docker containers and to separate configuration management task that are specific to Docker vs OS related.

Secure your systems

Change the generic OS build plan to include steps for customization to OS provisioning to secure your systems for production use. Include the following steps in your OS build plan:

- Update default passwords for root
- Change ssh access for direct root login
- Update yum repositories to trusted repositories
- Install docker CA certs for private registries for self-signed certs

Employ configuration management best practices

Create configuration management using SCM or the secure store repository for the `~/docker/machine` folder. Manage where you store and save credentials created during the docker-machine create process. This allows you to share the resources your developers use as a team and collaborate on those resources in a version-controlled manner.

Provision Docker host via DHCP

DHCP, with its dynamic IP address provisioning, is the preferred method for provisioning the Docker host. Static IP addresses add unnecessary complexities.

Run only one process per container

Run a single process in a single container. If a service depends on another service, Docker provides a mechanism for linking containers that allows sharing of information between linked source and recipient containers. Decoupling applications into multiple containers makes it easier to scale horizontally and reuse containers.

Avoid installing unnecessary packages

Keep things simple by installing only the packages you need. This reduces complexity, external dependencies, file sizes, and build sizes. Don't include a package just because it might be "nice to have." Stick to what's necessary.

Get a single-point-of-contact to support your Docker deployments

Hewlett Packard Enterprise offers a single point-of-contact for enterprise-grade support, providing an ownership experience that reduces complexity with one place to call for access to a global network of specialists, 24x7.

Summary

Today's Idea Economy is driving IT leaders to find new and innovative ways to deliver the flexibility of hybrid IT solutions while reducing complexity and operating costs. Composable Infrastructure provides the promise of allowing IT to provision and manage traditional workloads along with new mobile and cloud-native applications from a single infrastructure, allowing you to achieve the vision of infrastructure as code.

A key tool in providing infrastructure as code is Docker, which provides programmers, development teams, and operations engineers the common toolbox they need to take advantage of the distributed and networked nature of modern applications. HPE OneView, together with Docker, brings containerization out of the cloud and on to your bare-metal infrastructure, giving your DevOps teams true infrastructure as code capabilities within your data center and supporting faster time-to-value for new revenue generating applications.

Learn more at

hpe.com/info/composableprogram

hpe.com/info/oneview

docker.com



Sign up for updates

★ Rate this document



© Copyright 2015 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty. Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.

Oracle is a registered trademark of Oracle and/or its affiliates. The OpenStack Word Mark is either a registered trademark/service mark or trademark/service mark of the OpenStack Foundation, in the United States and other countries and is used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation or the OpenStack community. Pivotal and Cloud Foundry are trademarks and/or registered trademarks of Pivotal Software, Inc. in the United States and/or other countries. Linux is the registered trademark of Linus Torvalds in the U.S. and other countries. VMware is a registered trademark or trademark of VMware, Inc. in the United States and/or other jurisdictions.